

Linked List (Singly, Doubly, and Circular) —with Implementation

1. The struct in C++

A struct (short for 'structure') is a user-defined data type that groups several variables — possibly of different types — under a single name. Linked lists are built entirely out of structs, so understanding struct is the first step.

1.1 What is a struct and why do we need it?

- A single node in a linked list needs to hold two different kinds of information at once: the data value, and the memory address of the next node.
- A plain int or float cannot hold both at once — we need a container. struct is that container.
- Once declared, a struct behaves like a new data type: you can create variables of that type, arrays of it, or (most importantly for us) pointers to it.

1.2 Declaration syntax

```
struct Node {  
  
    int val;        // the data stored in this node  
  
    Node* next;     // pointer to the next node (address)  
  
};
```

- 'struct Node { ... };' declares a new type called Node.
- Inside the braces we list the members: val (an int) and next (a pointer to another Node).
- The semicolon after the closing brace is required.

1.3 Declaring / creating variables of struct type

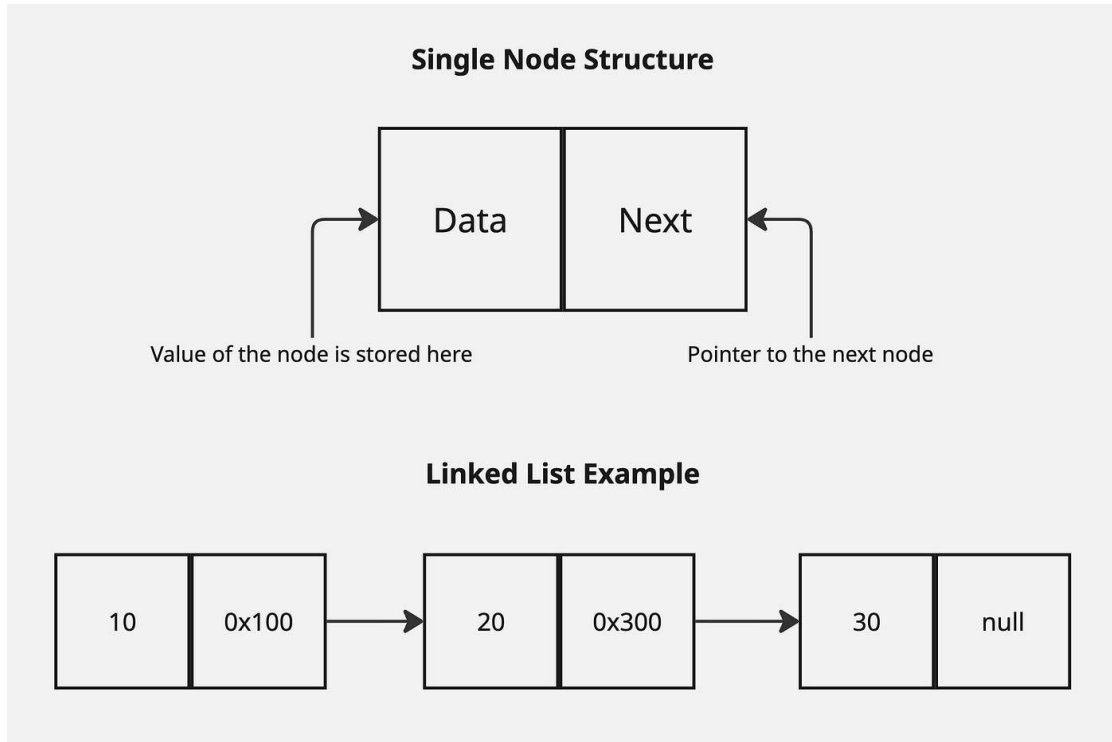
```
Node a;                // a single Node variable on the stack, can be accessed data via  
                        dot operation  
  
Node* head;            // a pointer to a Node (this is what we use for lists)  
  
Node* newNode = new Node(); // dynamically allocate a Node on the heap
```

1.4 Accessing / calling members

- Dot operator (.) — used when you have the actual struct variable: a.val = 25;
- Arrow operator (->) — used when you have a pointer to a struct: newNode->val = 25; This is shorthand for (*newNode).val = 25;
- In linked lists we almost always work with pointers, so -> is the operator you will use constantly.

1.5 Memory allocation

Every node of a linked list is created with 'new', which allocates memory on the heap at runtime and returns its address. This is what makes linked lists dynamic — unlike arrays, we don't need to know in advance how many nodes we will need.



```
Node* newNode = new Node(); // reserve memory for one Node, get back its address
newNode->val = 25;           // store data
newNode->next = nullptr;     // no next node yet
```

When a node is deleted, its memory must be released with `free()` or `delete`, otherwise the memory stays reserved forever (a memory leak).

2. Array — a quick recap

An array is the simplest way to store multiple values of the same type in contiguous memory locations.

Array (contiguous memory)



Linked List (scattered memory, connected by pointers)



Property	Array	Linked List
Size	Fixed at creation	Dynamic — grows/shrinks at runtime
Memory layout	Contiguous	Scattered, linked via pointers
Access	Random access ($O(1)$) via index	Sequential access only ($O(n)$)
Insert/Delete in middle	Expensive — needs shifting	Cheap — just re-link pointers
Extra memory per element	None	One (or two) pointers per node

3. Linked List — Overview

A linked list is a dynamic data structure made of nodes, where each node stores data plus a pointer (link) to another node. Because nodes are not stored contiguously, the only way to reach a node is by following the chain of pointers starting from the head.

3.1 Types of linked lists

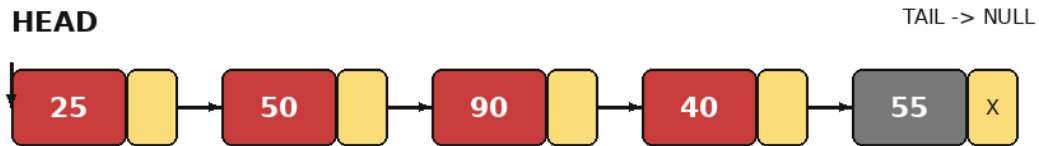
- Singly Linked List (SLL) — each node has a link only to its successor (next node).
- Doubly Linked List (DLL) — each node has links to both its successor (next) and its predecessor (prev).
- Circular Linked List (CLL) — a variation of either SLL or DLL where the last node points back to the head instead of to NULL, forming a circle. A circular list has no true 'end'.

3.2 Common list operations (covered for every type below)

- Create an empty list
 - Insert a node — at the beginning, at the end, or after a given node
 - Delete a node — from the beginning, from the end, or after a given node
 - Display / traverse all nodes
 - (Bonus) Detect a cycle in a list
-

4. Singly Linked List (SLL)

In an SLL, every node stores its data and a single pointer 'next' to the following node. The last node's next pointer is NULL, marking the end of the list.

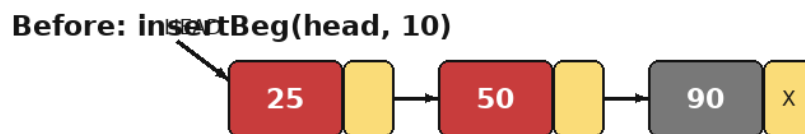


4.1 The struct definitions used throughout

```
struct Node {  
    int val;  
    Node* next;  
};
```

4.2 Insert at the beginning — insertBeg() (Time: $O(1)$)

Visual idea: create the new node, point its next to the current head, then move head to the new node. Nothing else needs to shift.



```
Node* insertBeg(Node *head, int val){ //O(1)  
    Node *newNode=new Node();  
    newNode->val=val;
```

```

newNode->next=head;

head=newNode;

return head;
}

```

4.3 Insert at the end — insertEnd() (Time: $O(n)$)

Visual idea: walk from head until you find the node whose next is NULL (the current last node), then attach the new node there.

Before: insertEnd(head, 60)



After: traverse to last node, link it to new node



```

Node *insertEnd(Node *head, int val){ //O(N)

    Node *ptr=new Node();

    ptr->val=val;

    ptr->next=nullptr;

    Node *t;

    t=head;

    if(t!=nullptr){

        while(t->next!=nullptr){

            t=t->next;

        }

        t->next=ptr;

    }

    else{

```

```

        head=ptr;

    }

    return head;
}

```

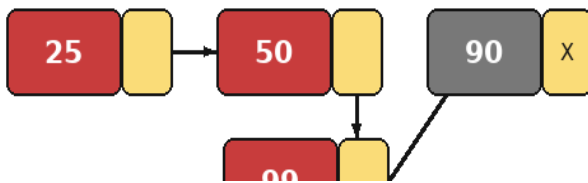
4.4 Insert after a given value — insert_after() (Time: O(n))

Visual idea: walk the list until you find the node whose value equals x, then splice the new node in right after it.

Before: insert_after(head, 99, x=50) -- insert 99 after node with val 50



After: node(50).next -> node(99); node(99).next -> node(90)



```

Node * insert_after(Node *head, int val,int x){ //O(N)

    Node *ptr, *newNode;

    newNode=new Node();

    ptr=head;

    newNode->val=val;

    while(ptr->val!=x && ptr!=nullptr){

        ptr=ptr->next;

    }

    if(ptr->val==x){

        newNode->next=ptr->next;

        ptr->next=newNode;

    }

}

```

```

    return head;
}

```

4.5 Display / traverse the list (Time: $O(n)$)

Visual idea: start a temporary pointer at head, print the data, move to next, and repeat until the pointer becomes NULL.

```

void display(Node *head){
    Node *ptr;
    ptr=head;
    if(ptr==nullptr){
        cout<<"LinkedList is empty"<<endl;
    }
    while(ptr!=nullptr){
        cout<<ptr->val<<" ";
        ptr=ptr->next;
    }
    cout<<endl;
}

```

4.6 Delete from the beginning — deleteBeg() (Time: $O(1)$)

Visual idea: keep a temporary pointer to the current head, move head to head->next, then free the old head.

Before: deleteBeg(head)



After: head = head->next; old head freed



```

Node *deleteBeg(Node *head){
    Node *ptr;
    if(head==nullptr){
        cout<<"List is empty"<<endl;
    }
    else{
        ptr=head;
        head=head->next;
        free(ptr);
    }
    return head;
}

```

4.7 Delete from the end — deleteEnd() (Time: $O(n)$)

Visual idea: walk to the second-last node (keeping a trailing pointer), disconnect the last node by setting the second-last node's next to NULL, then free the last node.

Before: deleteEnd(head)



After: second-last node's next set to NULL; last node freed



```

Node *deleteEnd(Node *head){ //O(N)
    Node *ptr, *prep;
    if(head==nullptr){
        cout<<"List is empty"<<endl;
    }
}

```

```

else if(head->next==nullptr){
    free(head);
    head=nullptr;
}
else{
    ptr=head;
    while(ptr->next!=nullptr){
        prep=ptr;
        ptr=ptr->next;
    }
    prep->next=nullptr;
    free(ptr);
}
return head;
}

```

4.8 Delete the node after a given value — deleteAfter() (Time: $O(n)$)

```

Node *deleteAfter(Node *head,int k){
    Node *ptr1,*ptr2;
    ptr1=head;
    while(ptr1->next!=nullptr){
        if(ptr1->val==k){
            ptr2=ptr1->next;
            ptr1->next=ptr2->next;
            free(ptr2);
            break;
        }
        ptr1=ptr1->next;
    }
}

```

```

    }

    return head;
}

```

4.9 Delete the entire list — deleteallnodes() (Time: $O(n)$)

```

Node *deleteallnodes(Node *head){ //O(N)

    Node *ptr=head;

    while(ptr!=nullptr){

        ptr=ptr->next;

        free(head);

        head=ptr;

    }

    return head;

}

```

4.10 Detecting a cycle — Floyd's algorithm (Time: $O(n)$)

Two pointers, 'slow' (moves 1 step) and 'fast' (moves 2 steps), traverse the list. If the list has a cycle, fast will eventually 'lap' slow and they will meet at the same node.

```

int isCycle(Node *head){

    if(head==nullptr || head->next==nullptr){

        return 0;

    }

    Node *slow=head;

    Node *fast=head;

    while(fast!=nullptr && fast->next!=nullptr){

        slow=slow->next;

        fast=fast->next->next;

        if(slow==fast){

```

```

        return 1;

    }

}

return 0;

}

```

4.11 Complete, runnable SLL program

```

#include<bits/stdc++.h>

using namespace std;

struct Node{

    int val;

    Node* next;

};

int main(){

    int n,x,y;

    cin>>n;

    Node *head;

    head=nullptr;

    for(int i=0;i<n;i++){

        int a;

        cin>>a;

        head=insertEnd(head,a);

    }

    display(head);

    cout<<"Enter a number which you want to insert and that number also after  
which you want to insert:"<<endl;

    cin>>x>>y;

```

```

    head=insert_after(head,x,y);

    display(head);

    head=deleteBeg(head);

    display(head);

    head=deleteEnd(head);

    display(head);

    int k;

    cout<<"Enter after which number's node you want to delete:"<<endl;

    cin>>k;

    head=deleteAfter(head,k);

    display(head);

    head=deleteallnodes(head);

    display(head);

    return 0;

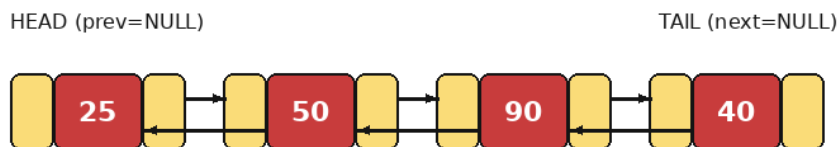
}

```

Note: place insertBeg, insertEnd, insert_after, display, deleteBeg, deleteEnd, deleteAfter, deleteallnodes, and isCycle (sections 4.2–4.10) above main() so the compiler recognizes them.

5. Doubly Linked List (DLL)

In a DLL, every node stores its data plus two pointers: 'next' (successor) and 'prev' (predecessor). This lets us traverse the list in both directions, at the cost of one extra pointer per node.



5.1 The struct definition

```

struct Node{
    int val;

    Node *prev;

    Node *next;
};

```

5.2 Insert at the beginning — insertBeg() (Time: $O(1)$)

Visual idea: same as SLL, but we must also set the old head's prev to point back to the new node, and the new node's prev to NULL.

Before: insertBeg(head, 10)



After: new node's next=old head; old head's prev=new node; head updated
HEAD (new)



```

Node *insertBeg(Node *head, int val){

    Node *newNode=new Node();

    newNode->val=val;

    newNode->prev=nullptr;

    newNode->next=head;

    if(head != nullptr)

        head->prev = newNode;

    head=newNode;

    return head;

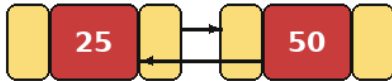
}

```

5.3 Insert at the end — insertEnd() (Time: $O(n)$)

Visual idea: walk to the last node ($\text{next} == \text{nullptr}$), attach the new node as its next, and set the new node's prev to point back to the old last node.

Before: insertEnd(head, 60)



After: traverse to last node; link new node as its next, set new node's prev



```
Node *insertEnd(Node *head, int val){
    Node *ptr,*newN;
    newN=new Node();
    newN->val=val;
    newN->next=nullptr;
    ptr=head;
    if(ptr!=nullptr){
        while(ptr->next!=nullptr){
            ptr=ptr->next;
        }
        ptr->next=newN;
        newN->prev=ptr;
    }
    else{
        newN->prev=nullptr;
        head=newN;
    }
    return head; }
```

5.4 Display / traverse the list (Time: $O(n)$)

```
void display(Node *head){  
    if(head==nullptr){  
        cout<<"List is empty"<<endl;  
    }  
    Node *ptr=head;  
    while(ptr!=nullptr){  
        cout<<ptr->val<<" ";  
        ptr=ptr->next;  
    }  
    cout<<endl;  
}
```

5.5 Delete from the beginning — deleteBeg() (Time: $O(1)$)

Visual idea: move head to head->next, set the new head's prev to NULL, then free the old head. If the list becomes empty, handle that case separately.

```
Node *deleteBeg(Node *head){  
    Node *ptr;  
    if(head==nullptr){  
        cout<<"List is empty"<<endl;  
        return head;  
    }  
    ptr=head;  
    head=head->next;  
    if(head!=nullptr)  
        head->prev=nullptr;  
    free(ptr);  
    return head; }
```

5.6 Delete from the end — deleteEnd() (Time: $O(n)$)

Visual idea: walk to the last node using next pointers, then use its prev pointer ($O(1)$ once found) to unlink it — no need for a separate trailing pointer like in SLL.

```
Node *deleteEnd(Node *head){

    if(head==nullptr){

        cout<<"List is empty"<<endl;

        return head;

    }

    if(head->next==nullptr){

        free(head);

        return nullptr;

    }

    Node *ptr=head;

    while(ptr->next!=nullptr){

        ptr=ptr->next;

    }

    ptr->prev->next=nullptr;

    free(ptr);

    return head;

}
```

5.7 Complete, runnable DLL program

```
#include<bits/stdc++.h>

using namespace std;

struct Node{

    int val;

    Node *prev;

    Node *next;
```

```

};

int main(){
    Node *head;

    head=nullptr;

    int n;

    cin>>n;

    for(int i=0;i<n;i++){
        int a;

        cin>>a;

        head=insertBeg(head,a);
    }

    display(head);

    return 0;
}

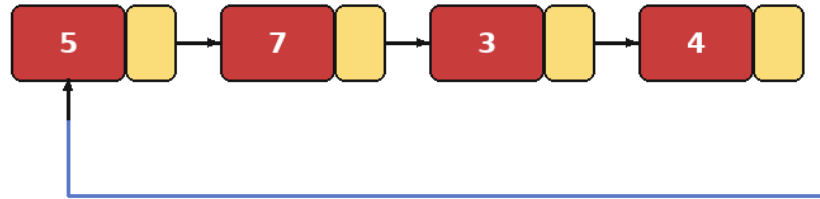
```

Note: place insertBeg, insertEnd, display, deleteBeg, and deleteEnd (sections 5.2–5.6) above main().

6. Circular Linked List (CLL)

A circular linked list is not a third independent structure — it is a singly (or doubly) linked list where the last node's next pointer points back to the head instead of NULL. This means there is no 'end' — traversal can go on forever unless you explicitly stop after one full loop. Below we build a circular SLL.

Circular Singly Linked List: last node points back to HEAD



6.1 The struct definition (same as SLL)

```
struct Node{  
    int val;  
    Node* next;    // for the last node, this points back to head, not nullptr  
};
```

6.2 Insert at the beginning — insertBeg() (Time: $O(n)$ to find the last node)

Visual idea: create the new node, point it to the old head, make it the new head, then walk around the circle to find the last node and re-point its next to the new head.

```
Node *insertBeg(Node *head, int val){  
    Node *newNode=new Node();  
    newNode->val=val;  
    if(head==nullptr){  
        newNode->next=newNode;    // points to itself  
        return newNode;  
    }  
    newNode->next=head;  
    Node *ptr=head;  
    while(ptr->next!=head){        // find the last node  
        ptr=ptr->next;  
    }  
    ptr->next=newNode;            // last node now points to the new head
```

```

    head=newNode;

    return head;
}

```

6.3 Insert at the end — insertEnd() (Time: $O(n)$)

Visual idea: walk the circle until you find the node whose next points back to head (the last node), insert the new node there, and make the new node point back to head.

```

Node *insertEnd(Node *head, int val){

    Node *newNode=new Node();

    newNode->val=val;

    if(head==nullptr){

        newNode->next=newNode;

        return newNode;

    }

    Node *ptr=head;

    while(ptr->next!=head){

        ptr=ptr->next;

    }

    ptr->next=newNode;

    newNode->next=head;

    return head;

}

```

6.4 Display / traverse the list (Time: $O(n)$)

Visual idea: start at head, print, move to next, and stop as soon as you arrive back at head — otherwise the loop never ends.

```

void display(Node *head){

    if(head==nullptr){

        cout<<"List is empty"<<endl;

    }

}

```

```

        return;
    }

    Node *ptr=head;

    do{

        cout<<ptr->val<<" ";

        ptr=ptr->next;

    } while(ptr!=head);

    cout<<endl;

}

```

6.5 Delete from the beginning — deleteBeg() (Time: $O(n)$)

Visual idea: find the last node (the one whose next is head), point it to head->next instead, then free the old head.

```

Node *deleteBeg(Node *head){

    if(head==nullptr){

        cout<<"List is empty"<<endl;

        return head;

    }

    if(head->next==head){        // only one node

        free(head);

        return nullptr;

    }

    Node *ptr=head;

    while(ptr->next!=head){

        ptr=ptr->next;

    }

    Node *newHead=head->next;

    ptr->next=newHead;

```

```

    free(head);

    return newHead;
}

```

6.6 Delete from the end — deleteEnd() (Time: $O(n)$)

Visual idea: walk with a trailing pointer until you find the last node ($\text{next} == \text{head}$), then link the second-last node directly to head.

```

Node *deleteEnd(Node *head){

    if(head==nullptr){

        cout<<"List is empty"<<endl;

        return head;

    }

    if(head->next==head){

        free(head);

        return nullptr;

    }

    Node *ptr=head, *prev=nullptr;

    while(ptr->next!=head){

        prev=ptr;

        ptr=ptr->next;

    }

    prev->next=head;

    free(ptr);

    return head;

}

```

6.7 Complete, runnable circular SLL program

```

#include<bits/stdc++.h>

```

```

using namespace std;

struct Node{
    int val;
    Node* next;
};

int main(){
    Node *head=nullptr;

    int n;
    cin>>n;

    for(int i=0;i<n;i++){
        int a;
        cin>>a;
        head=insertEnd(head,a);
    }

    display(head);

    head=insertBeg(head,10);

    display(head);

    head=deleteBeg(head);

    display(head);

    head=deleteEnd(head);

    display(head);

    return 0;
}

```

Note: place insertBeg, insertEnd, display, deleteBeg, and deleteEnd (sections 6.2–6.6) above main(). The same circular idea (last node points to head) can be applied to a doubly linked list to get a circular doubly linked list, where head->prev is also set to the last node.